

Tema 01. Introducción al aprendizaje Automático

Autor: Ismael Sagredo Olivenza

1. Qué es el aprendizaje automático (Machine Learning)

Aprendizaje

- El acto, proceso o experiencia de adquirir conocimiento y habilidades.

Automático

- Que trabaja por si mismo.

1.1 Definición de Machine Learning

Arthur Samuel (1959):

"Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed"

Tom Mitchell (1998)

"Well-posed Learning Problem: A computer program is said to learn from experience **E** with respect to some **task** (T) and some **performance** measure (P), if its performance on T, as measured by P, improves with **experience E**"

Tom M. Mitchell; Machine Learning; McGraw-Hill, 1998

Ejemplos

Reconocimiento de escritura

- T: Reconocer la escritura a partir de un bitmap
- P: Porcentaje de palabras identificadas
- E: Una colección de bitmaps con las palabras que representan.

Conducción autónoma

- T: Conducir un coche usando sensores de visión.
- P: Distancia recorrida sin cometer fallos
- E: Secuencia de imágenes junto a las direcciones de sus ruedas y la cantidad de aceleración o frenado realizado

1.2 ¿Cómo aprenden las máquinas?

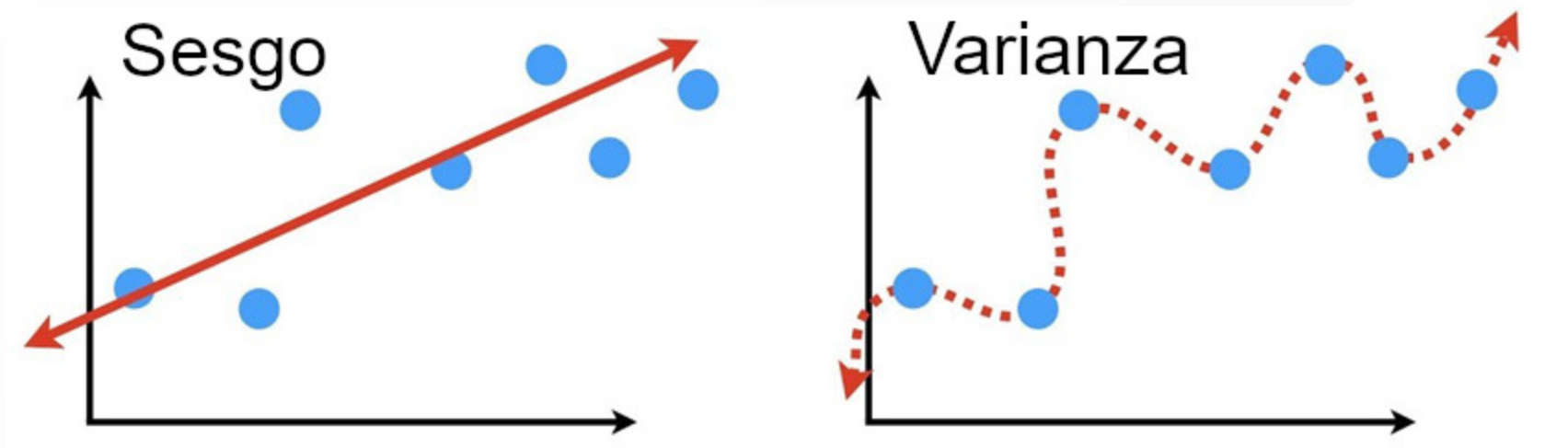
Se requiere una **abstracción del problema** para poder resolver nuevos ejemplos de dicho problema.

- Machine learning realiza **aprendizaje inductivo**, es decir, obtiene conclusiones en base a ejemplos.
- Abstrar genera **sesgos (bias)** y estos sesgos son necesarios para que el aprendizaje tenga éxito. Un modelo debe tener **Capacidad de generalización (generalizability)**

Los humanos introducimos sesgos en nuestro aprendizaje?

Por supuesto que si :)

- Reglas ortográficas (algunas de ellas tienen excepciones)
- Las ecuaciones físicas que estudiamos sobre el campo eléctrico o el movimiento uniformemente acelerado asumen que los cuerpos están en el vacío.
- La teoría de la relatividad de Einstein's ($E=mc^2$), solo se aplica a partículas con masa (el fotón tiene una fórmula más compleja).



Source <https://skillsfuturetv.com/wp-content/uploads/2020/05/maxresdefault-360.jpg>

Sesgo (Bias): trata de extraer el comportamiento general del

Varianza: si no se introduce sesgo, intentaremos captar todos los detalles del modelo, perdiendo capacidad de generalización.

(Overfitting / sobre adaptación)

1.3 Tipos de machine learning

- **Aprendizaje supervisado (Supervised Learning):** Los ejemplos para entrenar disponen del resultado esperado.
- **Aprendizaje no supervisado (Unsupervised learning):** Los datos no están etiquetados con el resultado esperado. Actualmente son los que están obteniendo los resultados más impresionantes como los modelos de IA generativa: <https://www.ibm.com/es-es/topics/generative-ai>
- **Aprendizaje por refuerzo (Reinforcement Learning / RL):** el feedback es recibido por el modelo a través del entorno (Variación: RL por retroalimentación humana (RLHF))

Tipos de problemas que podemos resolver

- **Clasificación:** identificar a qué categoría pertenece una instancia.
- **Regresión:** Predecir el valor real o continuo de una función
- **Clustering:** Cuando no conocemos las clases, podemos inferir cuantas y cuales son esas clases
- **Reducción de dimensión:** Extraer la información relevante de un conjunto de datos
- **Asociación:** Permite establecer relaciones entre variables (correlaciones)
- **Generación de contenidos:** Generación de imágenes, texto, video, gráficos.

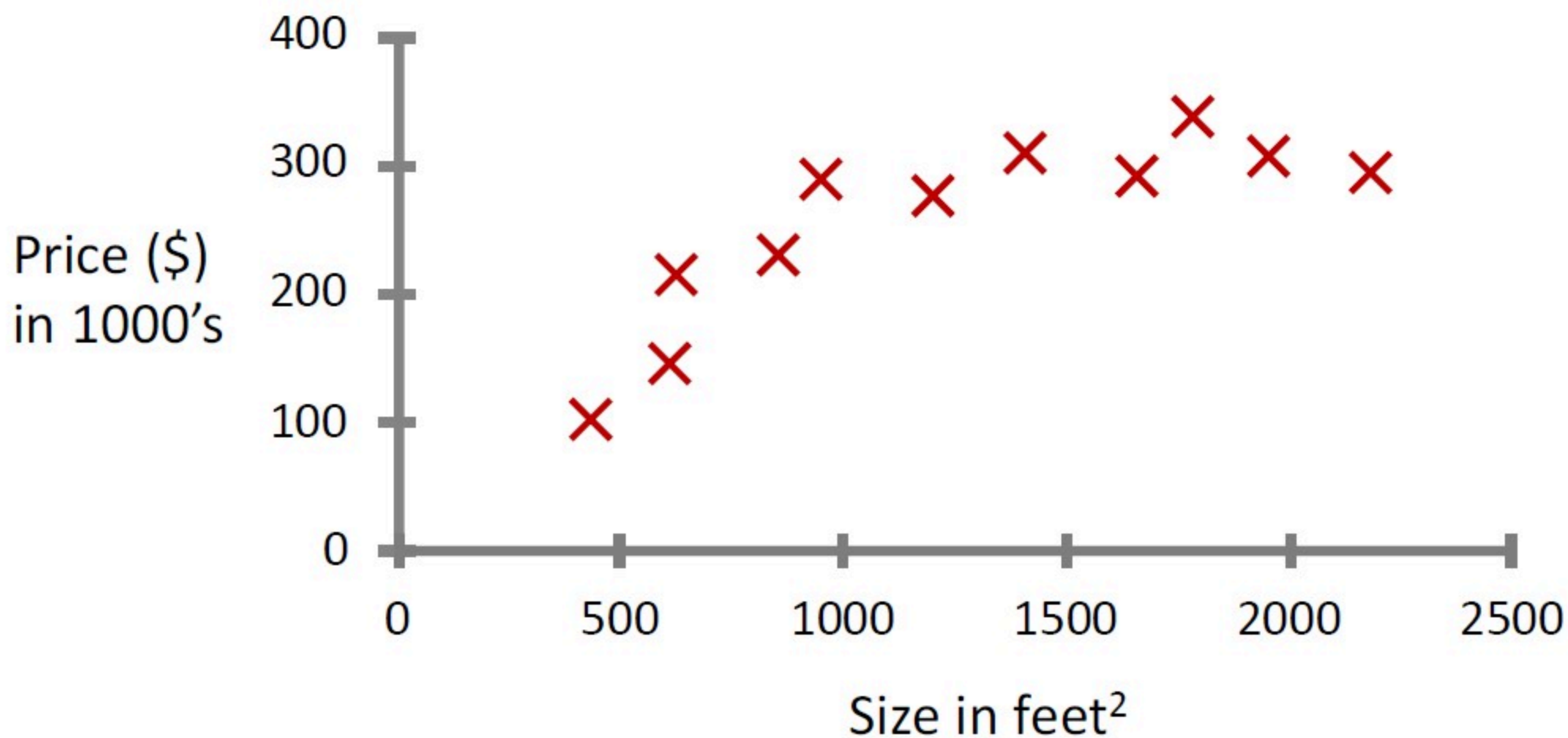
1.4 Aprendizaje supervisado

Aprende a partir de ejemplos con la **respuesta correcta**

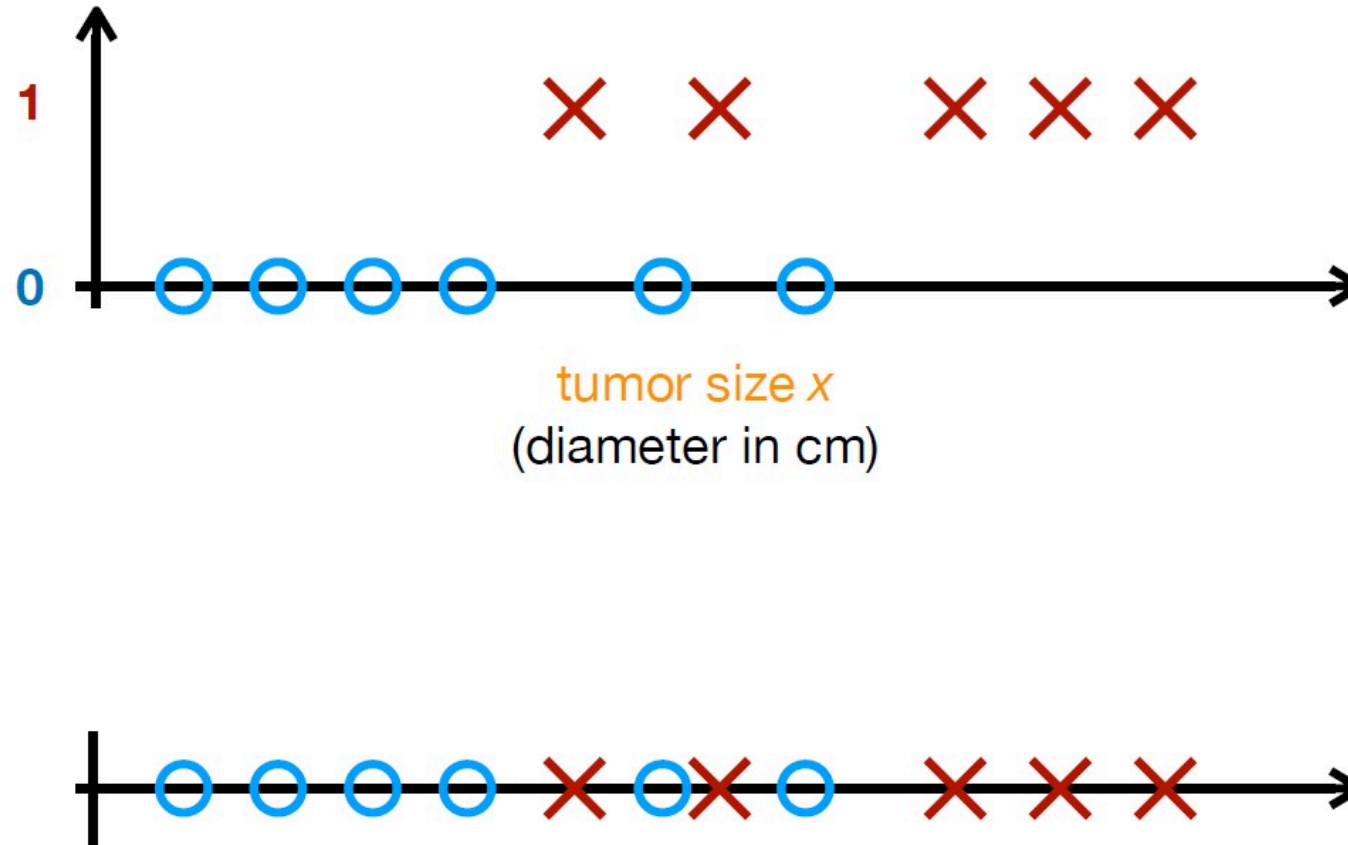
Weight	Height	Years	Years study	=	salary in thousands
20	170	30	10		30
27	175	35	5		25

- Ejemplo: Vector de N elementos. Cada elemento representa los atributos de una instancia
- Input: Una colección de M elementos (MxN)
- Output: Vector de M soluciones.

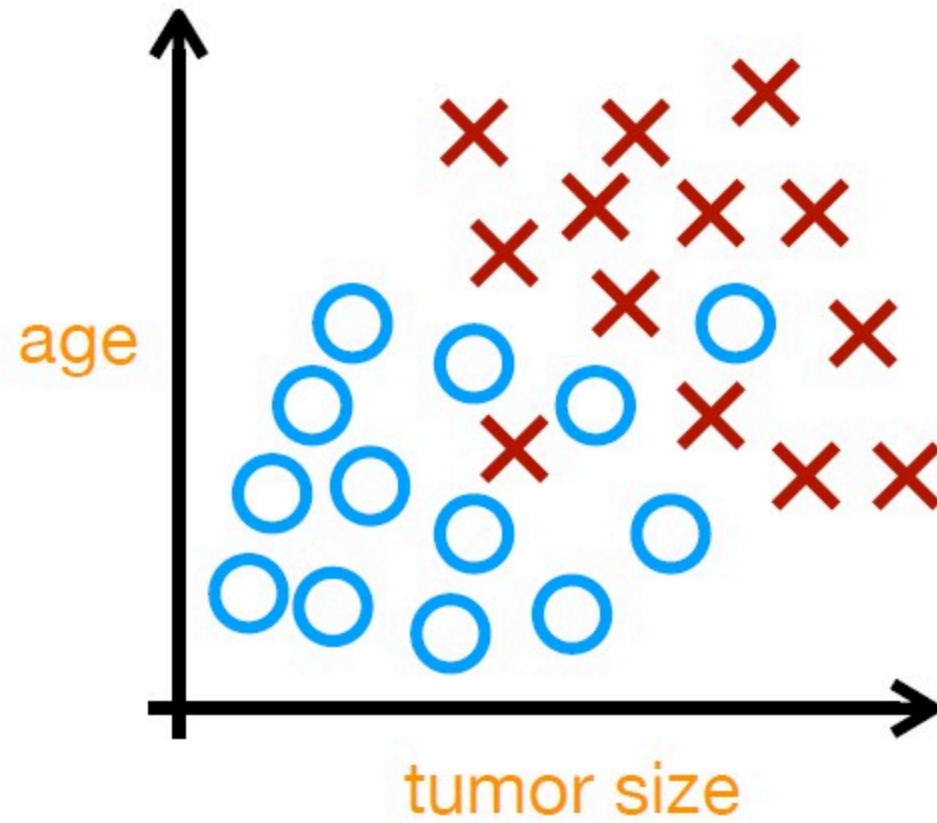
Regression: Housing price prediction



Classification: Breast cancer detection



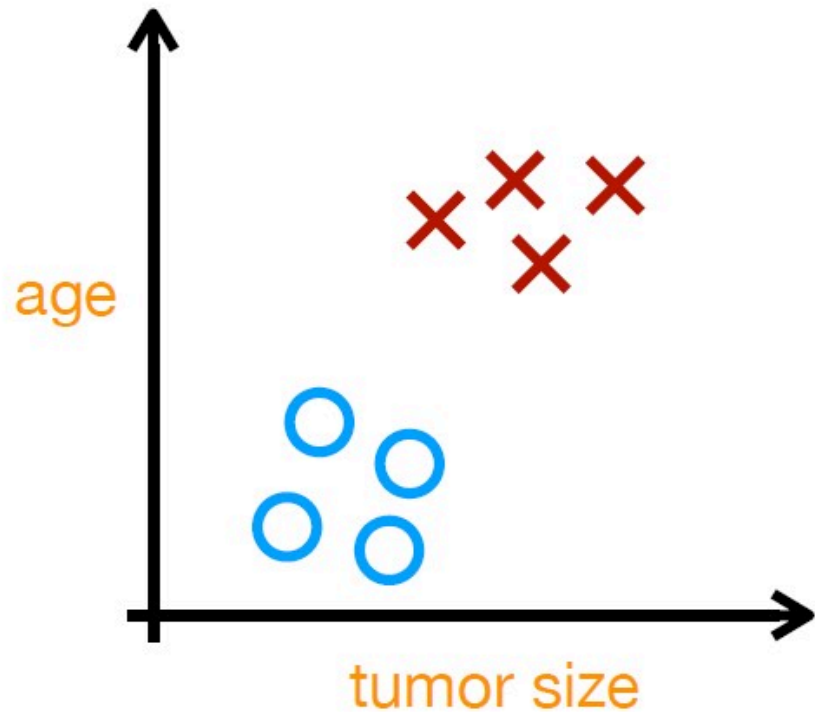
Two or more inputs



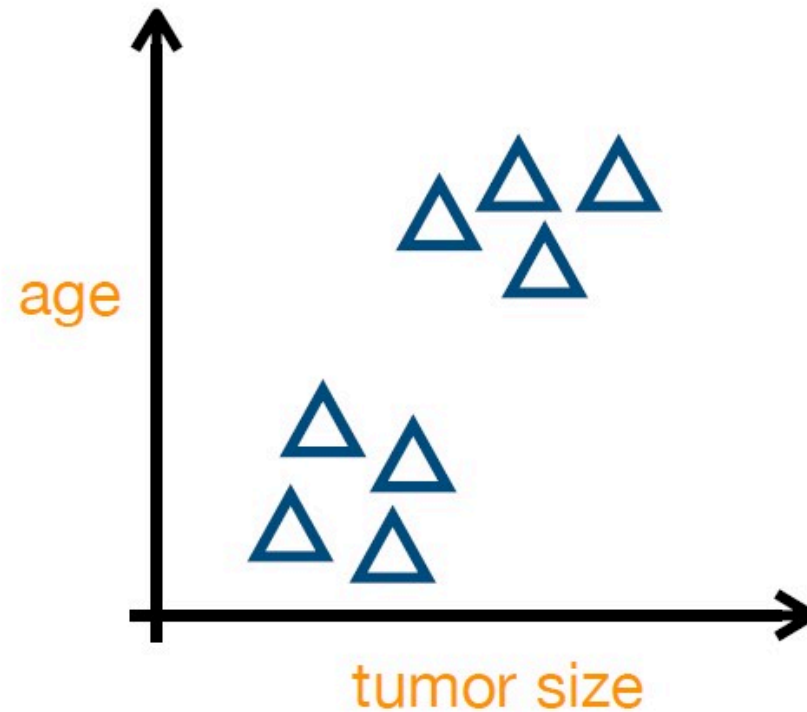
1.5 Aprendizaje no supervisado

- Si no conozco el valor que intentamos predecir en los ejemplos
- Extraer las características más relevantes de un conjunto de datos
- Normalmente necesitamos una interpretación humana para acotar el significado de los resultados obtenidos.
- IA generativa: puede generar contenido nuevo a partir de datos de entrada (Normalmente un prompt o pregunta en formato texto). Estos modelos son entrenados con datos sin etiquetar y generan **huecos ficticios** en los datos para aprender a predecir esos valores que faltan.

Supervised learning
Learn from data **labeled**
with the "**right answers**"



Unsupervised learning
Find something interesting
in **unlabeled** data



1.6 Aprendizaje por refuerzo (Reinforcement learning)

- El sistema aprende en modo online (no se basa en ejemplos previos)
- El entorno debe indicar al sistema si las acciones son correctas.
- Aprende por ensayo y error
- Especialmente adaptado a entornos dinámicos. El sistema se puede reciclar si cambia el entorno hasta cierto punto.
- Como la política que aprende no es del todo óptima, y explora caminos sub-óptimos es más robusto que otros métodos a cambios en el entorno (bueno para testing automático por ejemplo)

1.7 Ejemplos de ML en video juegos

- Pocos ejemplos de uso en el comportamiento de NPCs aunque existen. El problema es que los diseñadores no quieren que los modelos tengan alucinaciones que generen comportamientos emergentes no previstos.
- Pero se estan aplicando modelos de machine learning a la generación de imagen a mayor resolución, denoise para path-tracing, Anti Aliasing, animaciones, fisica del pelo, etc.
- Pero hay una activa investigación sobre ello.
- <https://www.ea.com/technology/research/ai-machine-learning>

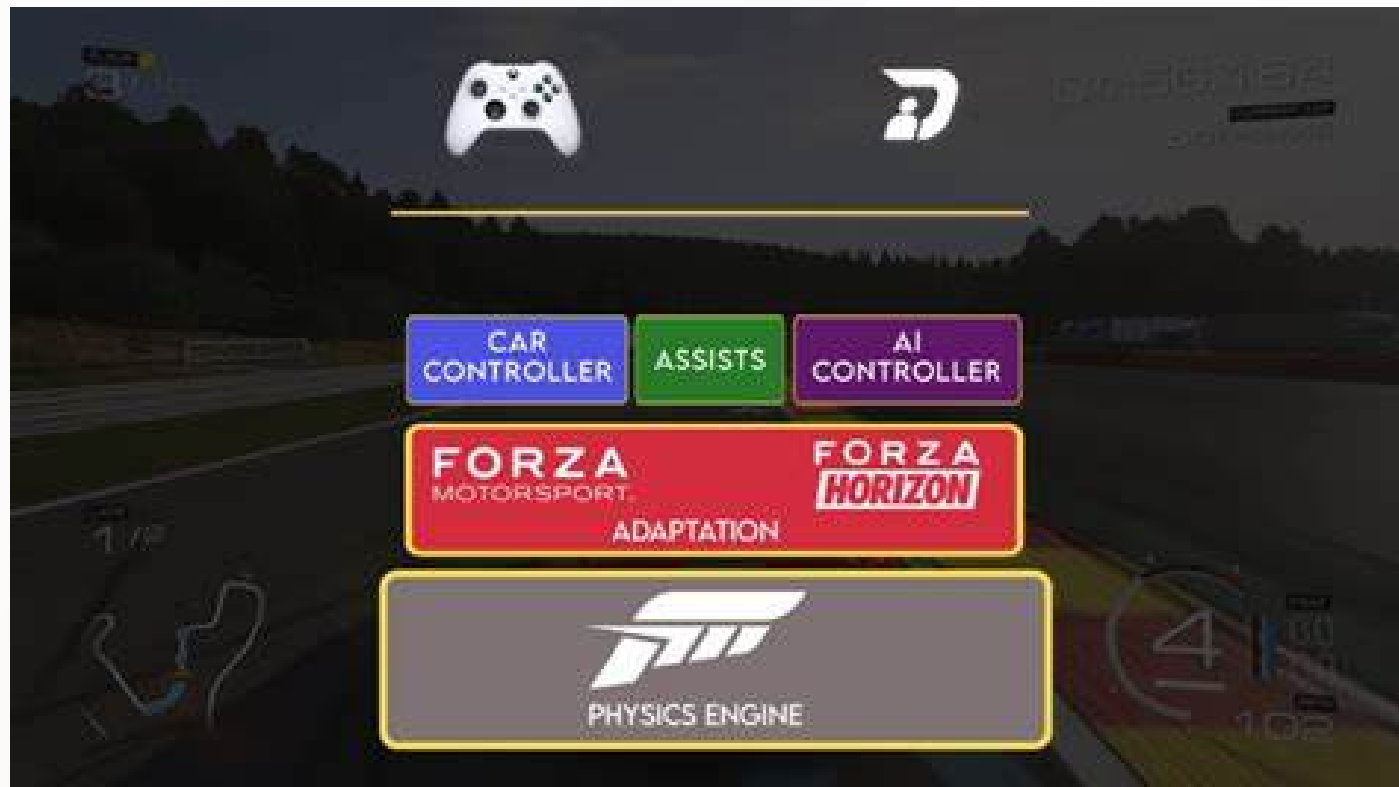
EA tiene algunos cursos de ML para desarrolladores de videojuegos.

Donde explican como implementar un perceptrón multicapa en C++ (nosotros lo haremos en Python)

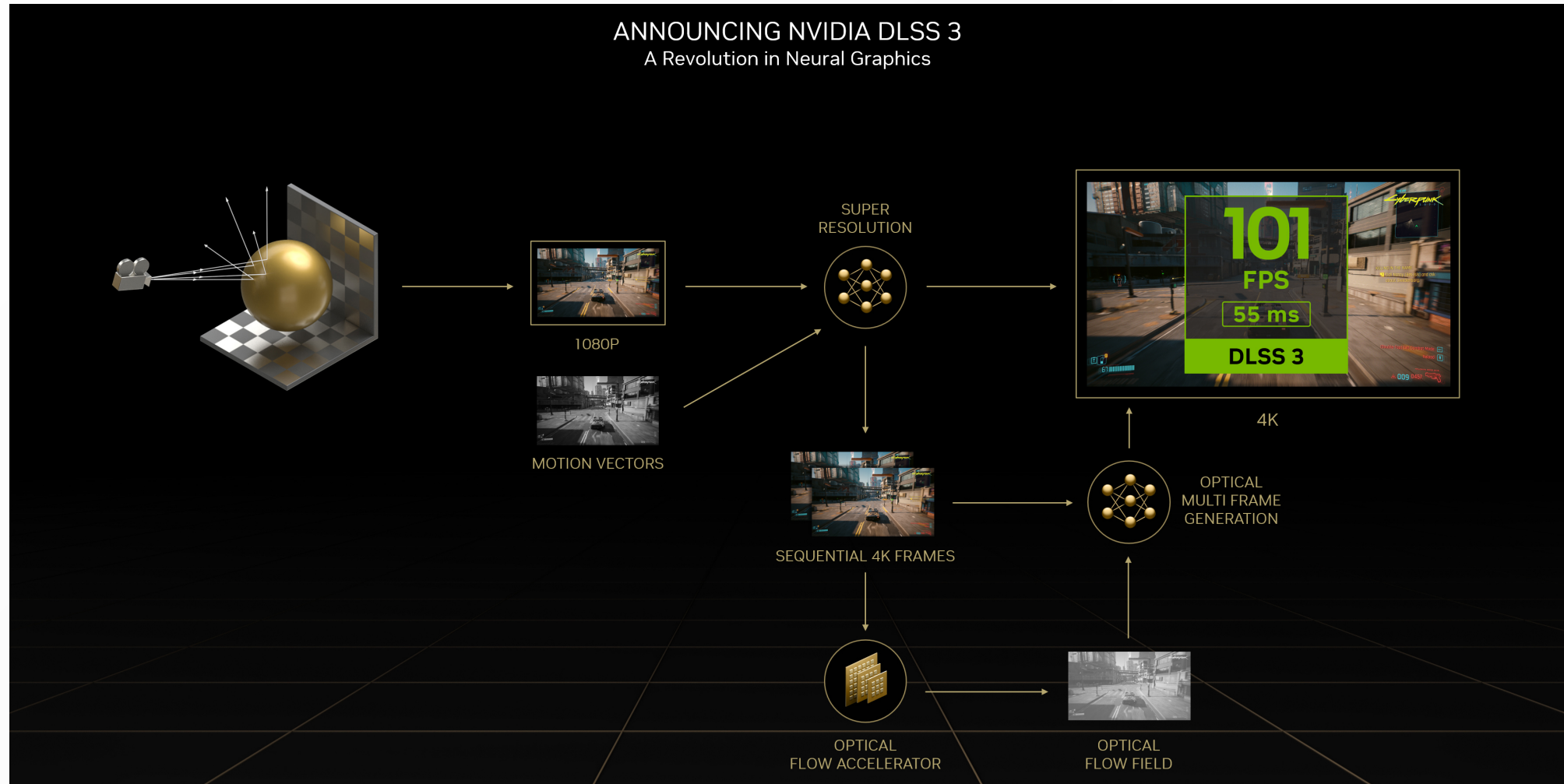
- Parte 1: <https://www.ea.com/technology/news/machine-learning-game-devs-part-1>
- Parte 2: <https://www.ea.com/technology/news/machine-learning-game-devs-part-2>
- Parte 3: <https://www.ea.com/seed/news/machine-learning-game-devs-part-3>

Forza Series: Drivatar

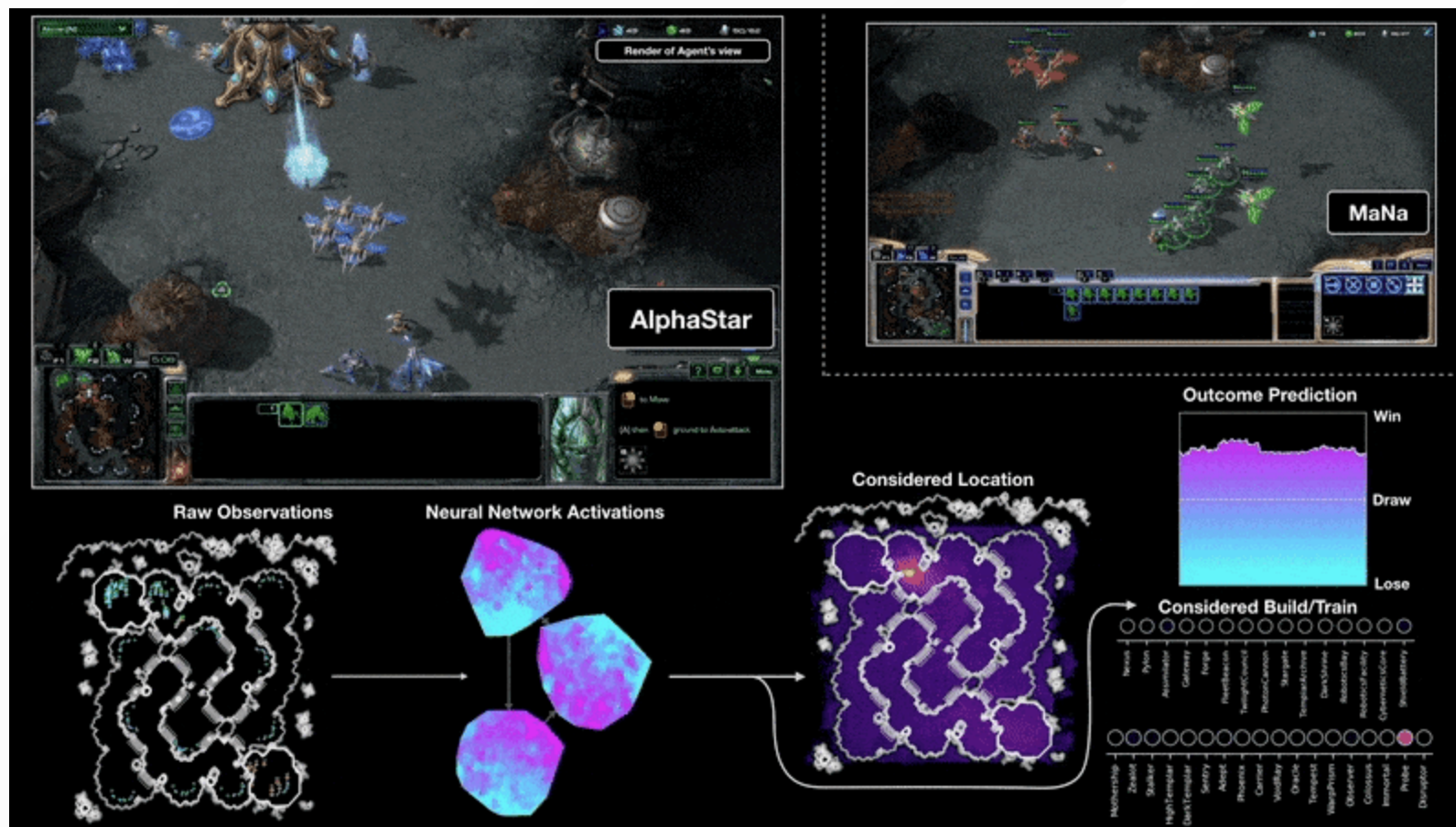
<https://web.archive.org/web/20070314015129/https://forzamotorsport.net/news/pitpassreports/pitpass36-2.htm>



Nvidia DLSS 3/4



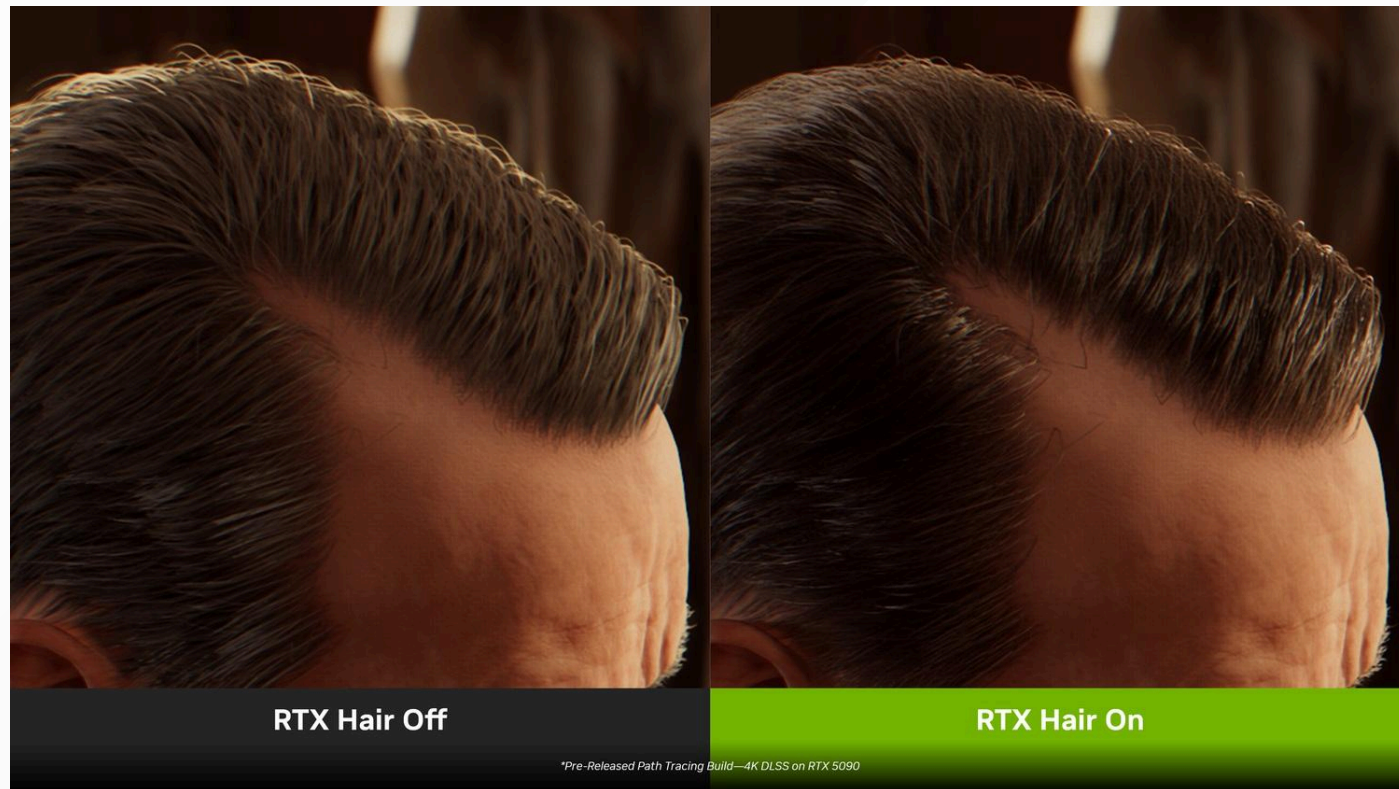
AlphaStar por Aprendizaje por refuerzo



<https://www.deepmind.com/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii>

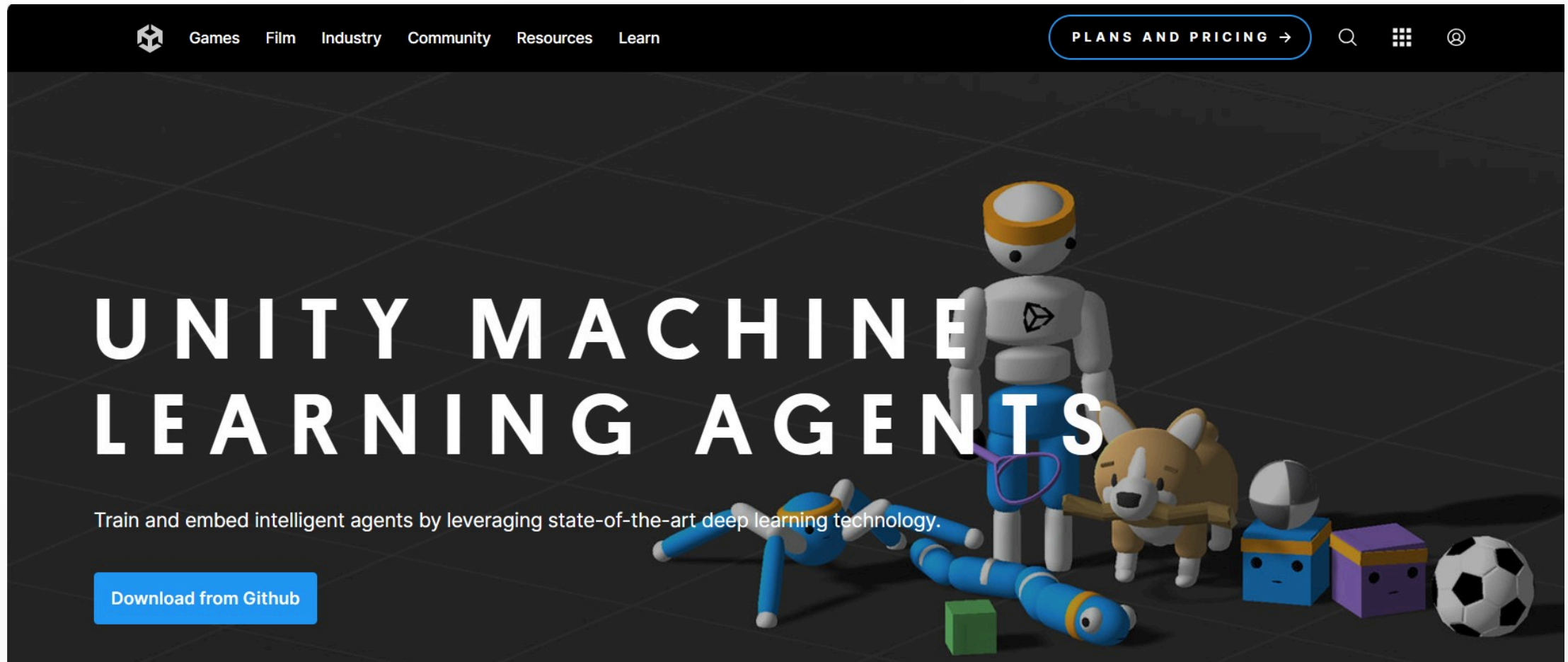
RTX Hair

<https://docs.unity3d.com/6000.1/Documentation/Manual/com.unity.ai.inference.html>



Unity Machine Learning Agents

<https://unity.com/products/machine-learning-agents>



Inference Engine (previamente denominado Unity Sentis)

<https://docs.unity3d.com/Packages/com.unity.ai.inference@2.2/manual/index.html>

<https://docs.unity3d.com/6000.1/Documentation/Manual/com.unity.ai.inference.html>

NN Engine in Unreal Engine

<https://dev.epicgames.com/documentation/en-us/unreal-engine/neural-network-engine-in-unreal-engine>

 Developer / Document... / Unreal Engine ▾ / Unreal Engine 5.4 Docum... / Making Interactive Expe... / Artificial Intelli... / Neural Network ...

Neural Network Engine

A collection of topics related to using artificial intelligence through Unreal Engine's neural network engine.



ON THIS PAGE

- Getting Started**
- Additional Topics
- Additional Resources

 Learn to use this **Beta** feature, but use caution when shipping with it.

The **Neural Network Engine (NNE)** provides a common API to access different neural network runtimes, and to evaluate neural networks without the need of runtime specific coding. NNE has various uses that can be applied to real-time inference to augment a game with artificial intelligence (AI) and to implement editor-based features as asset operations, queries, and artist-assisting tools.

Research:

<https://ieeexplore.ieee.org/abstract/document/8116624>

IEEE TRANSACTIONS ON GAMES, VOL. 11, NO. 1, MARCH 2019

5

Trained Behavior Trees: Programming by Demonstration to Support AI Game Designers

Ismael Sagredo-Olivenza , Pedro Pablo Gómez-Martín , Marco Antonio Gómez-Martín,
and Pedro Antonio González-Calero

Abstract—Programming by demonstration (PbD) has a straightforward application in the development of the artificial intelligence (AI) for nonplayer characters (NPCs) in a video game: a game designer controls the NPC during a training session in the game, and thus demonstrates the expected behavior for that character in different situations. Afterwards, applying some machine learning technique on the traces recorded during the demonstration, an AI for the NPC can be generated. Nevertheless, with this approach, it is very hard for the game designer to fully control the resulting behavior, which is a key requirement for game designers, who are responsible for putting together a fun experience for the player. In this paper, we present trained behavior trees (TBTs). TBTs are behavior trees (BTs) generated from traces obtained in a game through PbD. BTs are a technique widely used for AI game programming that are created and modified through special purpose visual editors. By inducing a BT from a PbD game session, we com-

such as “high general alertness (very nervous, thoroughly investigates every little disturbance, hard to circumvent, hard to shake off once he sees the target).”¹ Following the specifications given by designers, game AI programmers develop perception systems, behavior representation, path finding and decision making algorithms that conform with the AI system that game designers use to put together levels in the game.

AI game programmers and designers must work in close collaboration. Depending on the particular workflow of a given studio, designers may be in charge of just designing, tweaking parameters and testing the NPC behavior, or being responsible for programming part of the behavior through scripting, visual languages, or special purpose tools. Some solutions to AI problems are easy for a nontechnical designer but might take more

Reinforcement Learning Methods to Evaluate the Impact of AI Changes in Game Design

**Pablo Gutiérrez-Sánchez,¹ Marco A. Gómez-Martín,²
Pedro A. González-Calero,² Pedro P. Gómez-Martín²**

¹PadaOne Games, Calle Profesor Jose Garcia Santesmases, 9, 28040, Madrid, Spain

²Complutense University of Madrid, Madrid, Spain,

pablo.gutierrez@padaonegames.com, marcoa@fdi.ucm.es, pagoncal@ucm.es, pedrop@fdi.ucm.es

Abstract

Game development has become a long process that requires many professionals working on a project during several months or years. With this scenario the re-utilization of resources is crucial not only to alleviate the process but also to bring coherence into the final product. In this paper we focus on the reuse of NPCs and the problems it brings about. In particular it is common to have different breeds (or personalities) of NPCs that are placed on different levels on the game. The problem arises when their behaviors are fine-tuned to accommodate a specific level needs without taking into consideration that this change may alter their performance on previous already-tested levels. The paper presents the application of reinforcement learning together with behavior trees to automatically test if modifications to the AIs of a stealth game have an impact on the user experience. Our experiments re-

efforts of human testers towards less mechanical and more creative tasks.

In this paper we focus on automatic regression tests for detecting issues when modifications are made to AIs that are reused in multiple sections of the game. These changes may be done when fine-tuning the behavior of a NPC in a level without thinking about the implications of those small variations on other levels where the NPC was placed before.

Changes as simple as slightly modifying an enemy's movement speed could end up unexpectedly impacting how the player interacts with the game's levels. These changes need not be particularly drastic (such as sudden violations of the completeness of a level), and may boil down to the player taking more or less time to complete a part of the game, or exploiting a new way to beat a level that was